

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : WRITE A PROGRAM TO IMPLEMENT INFORMED SEARCH TECHNIQUE: AO* ALGORITHM USING PYTHON			
EXPERIMENT NO. : SSGMCE/WI/IT/6IT09/05		ISSUE NO. : 00	ISSUE DATE : 06-01-2025	
REV. DATE : NA		REV. NO. : 00	DEPTT. :	INFORMATION TECHNOLOGY
LABORATORY : COMPUTER SKILL LAB IV			SEMESTER : VI	PAGE : 1 OF 5

01 AIM : Write A Program To Implement Informed Search Technique: AO* Algorithm using Python

02 SCOPE : The experiment will be focusing on understanding the use of a AO* Algorithm as an example of informed search technique.

03 THEORY :

The AO* algorithm is an advanced search algorithm utilized in artificial intelligence, particularly in problem-solving and decision-making contexts. It is an extension of the A* algorithm, designed to handle more complex problems that require handling multiple paths and making decisions at each node.

The AO algorithm* (short for "And-Or Star") is a powerful best-first search method used to solve problems that can be represented as a directed acyclic graph (DAG). Unlike traditional algorithms like A*, which explore a single path, the AO* algorithm evaluates multiple paths simultaneously. This makes it more efficient for problems that involve AND-OR nodes. The AND nodes represent tasks where all child nodes must be satisfied, while OR nodes offer multiple alternative paths where only one child node needs to be satisfied to achieve the goal.

The AO* algorithm uses knowledge-based search, where both the start and target states are predefined. By leveraging heuristics, it identifies the best path, significantly reducing time complexity. Compared to the A algorithm*, AO* is more efficient when solving problems represented as AND-OR graphs, as it evaluates multiple paths at once.

This combination of best-first search, DAG representation, and heuristic optimization makes the AO algorithm* ideal for tackling complex problems that can be divided into interdependent tasks. Incorporating these techniques ensures faster and more effective problem-solving.

Working Principles of AO* Algorithm

The AO* algorithm works by utilizing a tree structure where each node represents a state in the problem space. The key components of the algorithm are:

Node Types

AND Nodes: Represent states where all child nodes must be satisfied to achieve a goal. If a task requires multiple conditions to be met, it would be represented as an AND node.

OR Nodes: Represent states where at least one child node must be satisfied to achieve a goal. This type is useful in scenarios where multiple paths can lead to a solution.

Heuristic Function

The algorithm employs a heuristic function, similar to A*, to estimate the cost to reach the goal from any given node. This function helps in determining the most promising paths to explore. The heuristic function

$h(n)$ estimates the cost to reach the goal from node n :

$h(n)$ =estimated cost to reach the goal from node

Search Process

The search begins at the initial node and explores its child nodes based on the type (AND or OR). The costs associated with nodes are calculated using the following principles:

For OR Nodes: The algorithm considers the lowest cost among the child nodes. The cost for an OR node can be expressed as:

$$C(n) = \min\{C(c_1), C(c_2), \dots, C(c_k)\}$$

where $C(n)$ is the cost of node n and $c_1, c_2, \dots, c_k$ are the child nodes of n .

For AND Nodes: The algorithm computes the cost of all child nodes and selects the maximum cost, as all conditions must be met. The cost for an AND node can be expressed as:

$$C(n) = \max\{C(c_1), C(c_2), \dots, C(c_k)\}$$

where $C(n)$ is the cost of node n , and $c_1, c_2, \dots, c_k$ are the child nodes of n .

Total Estimated Cost

The total estimated cost $f(n)$ at any node n is given by:

$$f(n) = C(n) + h(n)$$

where:

- $C(n)$ is the actual cost to reach node n from the start node.
- $h(n)$ is the estimated cost from node n to the goal.

In summary:

$$f(n) = \text{Actual cost} + \text{Estimated cost} \quad f(n) = \text{Actual cost} + \text{Estimated cost}$$

The search continues recursively until the goal is reached or all possibilities are exhausted.

Let us consider the below example:

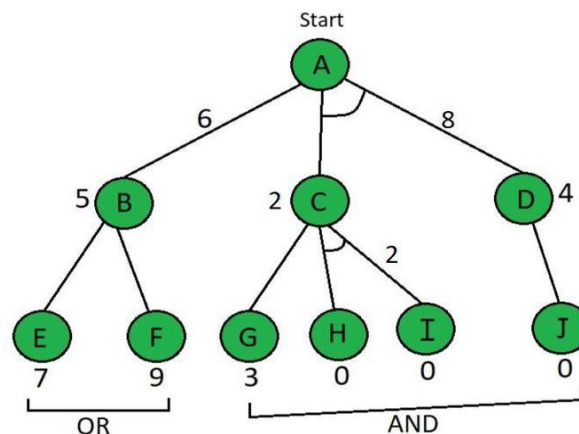


Fig: Weighted Acyclic Graph

04 PROGRAM

```
def Cost(H, condition, weight = 1):
    cost = {}
    if 'AND' in condition:
        AND_nodes = condition['AND']
        Path_A = ' AND '.join(AND_nodes)
        PathA = sum(H[node]+weight for node in AND_nodes)
        cost[Path_A] = PathA

    if 'OR' in condition:
        OR_nodes = condition['OR']
        Path_B = ' OR '.join(OR_nodes)
        PathB = min(H[node]+weight for node in OR_nodes)
        cost[Path_B] = PathB
    return cost

# Update the cost
def update_cost(H, Conditions, weight=1):
    Main_nodes = list(Conditions.keys())
    Main_nodes.reverse()
    least_cost = {}
    for key in Main_nodes:
        condition = Conditions[key]
        print(key, ': ', Conditions[key], '>>>', Cost(H, condition, weight))
        c = Cost(H, condition, weight)
        H[key] = min(c.values())
        least_cost[key] = Cost(H, condition, weight)
    return least_cost

# Print the shortest path
def shortest_path(Start, Updated_cost, H):
    Path = Start
    if Start in Updated_cost.keys():
        Min_cost = min(Updated_cost[Start].values())
        key = list(Updated_cost[Start].keys())
        values = list(Updated_cost[Start].values())
        Index = values.index(Min_cost)

        # FIND MINIMUM PATH KEY
        Next = key[Index].split()

        # ADD TO PATH FOR OR PATH
        if len(Next) == 1:
            Start = Next[0]
            Path += '<--' + shortest_path(Start, Updated_cost, H)
```

```

# ADD TO PATH FOR AND PATH
else:
    Path += '<--(' + key[Index] + ') '
    Start = Next[0]
    Path += '[' + shortest_path(Start, Updated_cost, H) + ' + '
    Start = Next[-1]
    Path += shortest_path(Start, Updated_cost, H) + ']'
return Path

# Example Usage:
H = {'A': -1, 'B': 5, 'C': 2, 'D': 4, 'E': 7, 'F': 9, 'G': 3, 'H': 0, 'I': 0, 'J': 0}
Conditions = {
    'A': {'OR': ['B'], 'AND': ['C', 'D']},
    'B': {'OR': ['E', 'F']},
    'C': {'OR': ['G'], 'AND': ['H', 'I']},
    'D': {'OR': ['J']}
}

# Weight
weight = 1

# Updated cost
print('Updated Cost:')
Updated_cost = update_cost(H, Conditions, weight)
print('* * 75)

# Shortest Path
print('Shortest Path:\n', shortest_path('A', Updated_cost, H))

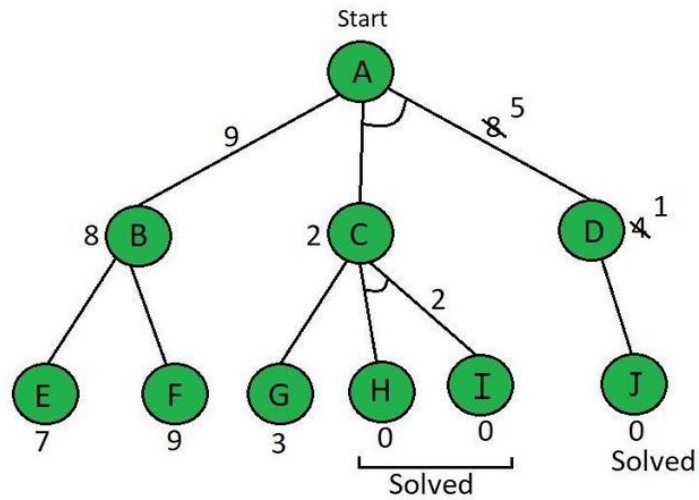
05 OUTPUT

Updated Cost:
D : {'OR': ['J']} >>> {'J': 1}
C : {'OR': ['G'], 'AND': ['H', 'I']} >>> {'H AND I': 2, 'G': 4}
B : {'OR': ['E', 'F']} >>> {'E OR F': 8}
A : {'OR': ['B'], 'AND': ['C', 'D']} >>> {'C AND D': 5, 'B': 9}
*****

Shortest Path:
A<--(C AND D) [C<--(H AND I) [H + I] + D<--J]

```

Solved Graph for the problem in the program will be as follows:



06 CONCLUSION

This is how we have successfully implemented the AO* algorithm to find the optimal path using heuristic value and cost update approach.